

MSCS-532  
Algorithms and Data Structures  
Spring 2026

Name: Victor Hugo Abad Navarrete  
Project Phase 2 : Proof of Concept Implementation  
University of the Cumberlands

## 1. Partial Implementation Overview

Phase two of the flight reservation and route optimization system implements the project's data structures and core functionality, and includes the first implementation of the algorithm that connects all the parts to calculate the most optimal flight route based on the user's preferences. The data structures and their methods were evaluated for correctness by a script that imports the created classes and verifies essential properties.

The problem statement is addressed by creating three data structures and a route calculation method. The data structures and route calculation method enable switching between different optimization targets at any point. The graph traversal algorithm requires minimal changes to achieve this, and the heap data structure is unchanged; only the weight is changed when switching (Cui, 2025).

### 1.1 Airport Registry

The AirportRegistry class wraps Python's built-in dict to provide a typed, documented interface for managing airport metadata. This data structure is required to get the complete information from an airport, either when processing the query from a user, or when displaying the data on the final route. Each entry maps an IATA code to a nested dictionary containing the airport's full name, city, and country. Python's dict method was chosen since, on average, it provides  $O(1)$  complexity for insertion, deletion, and lookup (Guido Van Rossum, 2009). Besides insertion and removal, this class provides a method to get all the IATA codes matching for an airport name (`get_codes_by_name`) and a method to get the IATA codes matching for a city name (`get_codes_by_city`).

```
class AirportRegistry:
    """
    Hash table that maps IATA airport codes to airport metadata.
```

```

Keys   : IATA code (str), e.g. "00SC"
Values : list of metadata fields (name, city, country)

"""

def __init__(self):
    self._table: dict[str, tuple[str, str, str]] = {}

#Add new airport or update existing one

def add_airport(self, code: str, name: str, city: str, country: str) -> None:
    """Register an airport
    If the code already exists, it will be overwritten with the new metadata.
    Make sure to use uppercase IATA codes for consistency.
    """
    self._table[code.upper()] = (name, city, country)

#Remove airport by code

def remove_airport(self, code: str) -> bool:
    """Remove an airport by code. Returns True if it existed."""
    return self._table.pop(code.upper(), None) is not None

#Lookup by code
#Might return the metadata tuple (name, city, country) or None if the code is not found

def get_airport(self, code: str) -> Optional[dict]:
    """Lookup by IATA code. Returns None if not found."""
    return self._table.get(code.upper())
def get_codes_by_name(self, query: str) -> list[tuple[str, str, str, str]]:
    """
    Reverse lookup: regex query -> list of matching IATA codes from airport name
    """
    matches = []
    for code, info in self._table.items():
        name = info[0]
        if re.search(query, name, re.IGNORECASE):
            matches.append((code, info[0], info[1], info[2]))
    return matches

```

## 1.2 FlightGraph

The FlightGraph class implements the adjacency-list structure defined in Phase 1. This is implemented as a dictionary in Python. The key for each node is the airport code, and the value is a list containing the edges that connect this airport to a destination. Each edge (FlightEdge) is represented by a class that contains the destination ID, travel time, cost, and flight ID for that edge. The edge class returns the weight (time, price, number of connections) based on the query

parameter. FlightGraph includes methods to add edges, remove edges, and return all the airports that are connected to a node (neighbors).

## FlightEdge

```
class FlightEdge:
    """
    Represents a destination airport

    destination : IATA code of the arrival airport
    travel_time : flight duration in minutes
    cost : one-way ticket price in USD
    flight_id : unique flight identifier
    """

    __slots__ = ("destination", "travel_time", "cost", "flight_id")

    #Create node on initialization, no overloads

    def __init__(self, destination: str, travel_time: int,
                  cost: float, flight_id: str):
        self.destination = destination
        self.travel_time = travel_time # minutes
        self.cost = cost # USD
        self.flight_id = flight_id

    #Return weight based on optimization mode

    def weight(self, mode: str) -> float:
        """
        Return the weight based on cost function.

        time -> minimize total flight minutes
        cost -> minimize total ticket price (USD)
        connections -> minimize number of connections (weight = 1 per hop)
        """
        if mode == "time":
            return float(self.travel_time)
        if mode == "cost":
            return float(self.cost)
        if mode == "connections":
            return 1.0
        raise ValueError(f"Unknown optimization mode: '{mode}'. "
                        f"Choose 'time', 'cost', or 'connections'.")
```

## FlightGraph

```
class FlightGraph:
    """
    Weighted directed graph representing the airline route network.

    Representation : adjacency list
```

```
_adj[origin_code] = [FlightEdge, FlightEdge, ...]
```

Each FlightEdge stores TWO weights (travel\_time and cost) so that the same graph structure can serve all three optimization modes without duplication.

```
"""
```

```
def __init__(self):
    self._adj: dict[str, list[FlightEdge]] = {}
    self._edge_count: int = 0
```

```
#add airport node
```

```
def add_node(self, code: str) -> None:
    """Register an airport node. No-op if already present."""
    code = code.upper()
    if code not in self._adj:
        self._adj[code] = []
```

```
#Add directed edge between airports
```

```
def add_edge(self, origin: str, destination: str,
              travel_time: int, cost: float, flight_id: str) -> None:
    """
    Add a directed flight edge origin -> destination.
    creates nodes for both airports if not already present.
    """
    origin = origin.upper()
    destination = destination.upper()
    self.add_node(origin)
    self.add_node(destination)
    edge = FlightEdge(destination, travel_time, cost, flight_id)
    self._adj[origin].append(edge)
    self._edge_count += 1
```

```
#Remove edge by flight id
```

```
def remove_edge(self, origin: str, flight_id: str) -> bool:
    """Remove the first edge with the given flight_id from origin."""
    origin = origin.upper()
    edges = self._adj.get(origin, [])
    for i, e in enumerate(edges):
        if e.flight_id == flight_id:
            edges.pop(i)
            self._edge_count -= 1
            return True
    return False
```

```
#Return all edges from a given airport node
```

```
def neighbors(self, code: str) -> list[FlightEdge]:
    """Return all outgoing FlightEdge objects from the given airport."""
    return self._adj.get(code.upper(), [])
```

## 1.3 MinHeap

The MinHeap data structure implements a MinHeap. This data structure is the core of the route calculation algorithm. Each element of the MinHeap corresponds to a graph path being evaluated from the origin airport. The heap is implemented as a list of tuples. Each tuple contains the set weight, airport\_code, path. The weight is the accumulated weight on this path. The airport\_code is the last node of the path. The path contains the list of all the airports (nodes) visible by this path. The MinHeap structure contains the function decrease\_key (maintain the heap property when a key value increases), insert, extract\_min (return min value and delete it from the heap), increase\_key, is\_empty, and heapify (to restore heap property starting from a node).

```
class MinHeap:
    """
    Min-heap priority queue for route calculation.
    Each element is a tuple of (weight, airport_code, path).
    The heap is implemented as a list, and the hash table maps airport codes to their index in
    the heap for O(1) access during key updates.
    """
    def __init__(self, initial_data=None):
        self.heap = [] #List of tuples (weight, airport_code, path)
        self.size = 0
        #If initial data is provided, insert it into the heap
        #initial_data is a list of tuples (weight, airport_code, path)
        if initial_data is not None:
            for element in initial_data:
                self.insert(element[0], element[1], element[2])

        #Decrease key function: Update the weight of an element in the heap and maintain the heap
        #priority
        def decrease_key(self, index, new_weight):
            #Update the weight in the heap array
            airport_code = self.heap[index][1]
            path = self.heap[index][2]
            self.heap[index] = (new_weight, airport_code, path)
            while index > 0:
                parent_index = (index - 1) // 2
                if self.heap[index][0] < self.heap[parent_index][0]:
                    #Swap the current element with its parent
                    tmp_val = self.heap[index]
                    self.heap[index] = self.heap[parent_index]
                    self.heap[parent_index] = tmp_val
                    #Update the index to the parent's index for the next iteration
                    index = parent_index
                else:
                    break

            #insert function: First insert it in the heap array and return the index, then maintain
            #the heap property by decreasing the key
```

```

    # new element is always inserted at the end of the heap array, then we decrease the key to
    maintain the heap property
    def insert(self, weight, airport_code, path):
        #Insert the weight, airport_code, path tuple into the heap array
        #If the heap size is greater than self.size, it means that we may have some empty
        slots in the heap array due to previous extractions, so we can reuse those slots instead of
        appending to the end of the array
        if self.size < len(self.heap):
            self.heap[self.size] = (weight, airport_code, path)
        else:
            self.heap.append((weight, airport_code, path))
        self.size += 1
        #Decrease the key of the new element to maintain the heap property
        self.decrease_key(self.size - 1, weight)

    #extract min function: Remove and return the element with the smallest weight (the root of
    the heap), then maintain the heap property by decreasing the key of the last element and
    heapifying the root
    # Return the airport code, weight, and path of the extracted element
    def extract_min(self):
        if self.size == 0:
            return None #Heap is empty
        min_element = self.heap[0]
        weight, airport_code, path = min_element[0], min_element[1], min_element[2]
        #Make the weight of the root element positive infinity and swap it with the last
        element in the heap array, then heapify the root to maintain the heap property
        self.heap[0] = (float('inf'), airport_code, path)
        #Swap the root with the last element in the heap array
        tmp_val = self.heap[0]
        self.heap[0] = self.heap[self.size - 1]
        self.heap[self.size - 1] = tmp_val
        #Heapify the root to maintain the heap property
        self.size -= 1
        self.heapify(self.heap, self.size, 0)
        return airport_code, weight, path #Return the airport code, weight, and path of the
        extracted element

    #increase key function: Update the weight of an element in the heap and maintain the heap
    priority
    def increase_key_element(self, index, new_weight):
        #Update the weight in the heap array
        airport_code = self.heap[index][1]
        path = self.heap[index][2]
        self.heap[index] = (new_weight, airport_code, path)
        #Heapify the element at the given index to maintain the heap property
        self.heapify(self.heap, self.size, index)

    #decrease key function: Update the weight of an element in the heap and maintain the heap
    priority
    def decrease_key_element(self, index, new_weight):
        #Update the weight in the heap array
        airport_code = self.heap[index][1]
        path = self.heap[index][2]
        self.heap[index] = (new_weight, airport_code, path)
        #Decrease the key of the element at the given index to maintain the heap property
        self.decrease_key(index, new_weight)

    #Check if the heap is empty
    def is_empty(self):
        return self.size == 0

```

```

#Heapify function: Maintain the heap property by heapifying the element at the given index
def heapify(self, arr, n, i):
    #Start with i as the smallest element (the element to be heapified)
    smallest = i
    l = 2 * i + 1 # left = 2*i + 1
    r = 2 * i + 2 # right = 2*i + 2

    # If left child is smaller than root
    if l < n and arr[l][0] < arr[smallest][0]:
        smallest = l

    # If right child is smaller than smallest so far
    if r < n and arr[r][0] < arr[smallest][0]:
        smallest = r

    # If smallest is not root
    if smallest != i:
        # Swap elements in heap
        tmp_val = arr[i]
        arr[i] = arr[smallest]
        arr[smallest] = tmp_val

        #smallest keeps the swapped element's index, so we need to heapify the affected
sub-tree
        self.heapify(arr, n, smallest)

```

## 2. Demonstration and Testing

The functionality of the data structure was tested by two mechanisms. First, an initial implementation of the route calculation algorithm was implemented. This had the purpose of integrating all the elements together and performing real operations in the data structures to evaluate their functionality. Dijkstra's algorithm was implemented to find the shortest-cost path from the origin to the destination (Ibrahim Alameri, 2024). The entire data structure creation, manipulation, querying, and analysis was tested by creating the frontend code that consumes an airport database and a flight routes database, and asks the user for origin, destination, and a variable to optimize (time, cost, connections). The test with a reduced database yielded good results. All the routes generated by the program were manually reviewed for correctness.

```

py --airports airports_small.csv --flights routes_small.csv
Enter the origin airport (code, name, or city): NGX
Selected origin airport: Manang Airport - Ngawal, NP, NGX
Enter the destination airport (code, name, or city): YAN
Multiple airports found matching 'YAN':
1. YAN - Yangambi Airport, Yangambi, CD
2. YAN - Yangambi Airport, Yangambi, CD
3. YAN - Yangambi Airport, Yangambi, CD
Select the number corresponding to the correct destination airport: 1
Selected destination airport: Yangambi Airport - Yangambi, CD, YAN
Select what you prefer to prioritize (time, cost, connections): cost
Optimal route from 'NGX' to 'YAN' minimizing total cost:
Total cost: $1372.09
Travel summary:
NGX - Manang Airport, Ngawal, NP
RAJ - Rajkot Airport, Rajkot, IN
YAN - Yangambi Airport, Yangambi, CD
Do you want to make another query? (yes/no): 

```

**Figure 1. Sample route querying on the first implementation**

Second, a script that includes all the data structures was created, and basic properties of the structures were evaluated. The script checks for coherent object insertion, removal, and querying. It also checks for edge cases such as querying an invalid IATA code. All the checks passed.

### 3. Implementation Challenges and Solutions

The definition of the MinHeap data structure required evaluating various possibilities on which variables to include, along with the priority key (accumulated weight). Initially, an attempt was made to create a separate structure that identifies all the traversed paths, and for each of them, it saves the route taken. However, while implementing the MinHeap structure, this would have required many queries to this structure and updating it frequently. Hence, to avoid this, it was decided to include the path and the last node in the path as additional information in each MinHeap element. The rationale for this is that for objects that are members of a list, Python handles the objects by reference, so swapping elements is not expected to be heavy in memory traffic.

### 4. Next steps

In the last phase, all the methods implemented for the data structure will be reviewed to determine their complexity and find inefficiencies in the original design. This will include hardware profiling to identify potential issues with the data access pattern. Also, the route calculation algorithm will be revised for two purposes. First, to evaluate the implementation of

other graph traversing algorithms. Second, to allow returning more than one alternative to the user. In a real-world flight scheduling system, the user is presented with several alternatives to choose from, sorted from the most optimal one to the least optimal one.

## **5. References**

- Cui, T. S. (2025). Practice of an improved many-objective route optimization algorithm in a multimodal transportation case under uncertain demand. *Complex Intell. Syst.*, 136.
- Guido Van Rossum, F. L. (2009). *Python 3 Reference Manual: Python Documentation Manual*. SoHo Books.
- Ibrahim Alameri, J. K.-H. (2024). Optimizing Connections: Applied Shortest Path Algorithms for MANETs. *Computer Modeling in Engineering & Sciences*, 787-807.